

Finance OS / APAR

Accounts Payable & Receivable — autonomous, supervised by exception

Product Overview & User Guide

Version 2.0 · research build · June 2026

An AI/agentic finance back office: software agents ingest invoices and payments, perform 3-way matching, apply incoming cash, drive collections, and flag anomalies — while a human analyst supervises by exception rather than doing manual data entry.

1 · Overview

The problem

Accounts Payable and Receivable are high-volume, rules-heavy, and audited. A mid-size company processes tens of thousands of invoices a month — today largely by hand: keying data, reconciling spreadsheets, three-way-matching, chasing overdue accounts. It is slow, error-prone, and where money quietly leaks through duplicate payments and missed terms.

What Finance OS does

Finance OS turns that back office into a supervised, agentic system. Each workflow has a dedicated agent that does the routine work and escalates only what needs judgment. The analyst — our persona, Dana Okafor — reviews exceptions and tunes the agents instead of processing every line.

Principles

- **Supervise by exception.** Agents auto-handle the clean majority; humans see only what's ambiguous or high-value.
- **Deterministic where it matters, AI where it helps.** Rules do the math (matching, reconciliation); a language model only explains the gray-area cases — it never approves a payment.
- **Every action gated & logged.** Auto-actions fire only above an agent's confidence threshold; everything lands in an immutable audit trail with its reason and source documents.
- **Your data stays yours.** The reasoning model can run locally (Ollama / vLLM); connectors are vendor-swappable; the system runs offline on a mock seam.

Who it's for

AP/AR analysts and controllers in finance teams; the platform is generic and resellable to any organization once hosted.

2 · Guided walkthrough

The product is seven screens grouped as Overview, Accounts Payable, Accounts Receivable, and Autonomy. Each runs on live, persisted, agent-computed data.

2.1 Command Center

The Command Center opens with fleet-level metrics and an **Agents at work** strip showing every agent live, its mode and threshold, and today's touchless rate. AP and AR health tiles and a 30-day cash-flow forecast give an at-a-glance read of the back office.

Screenshot placeholder
save `demo/screens/01-command-center.png`

The home view: outcome KPIs a CFO cares about (touchless rate, open exceptions, close cycle, cash position, leakage recovered).

2.2 Exception Queue (AP)

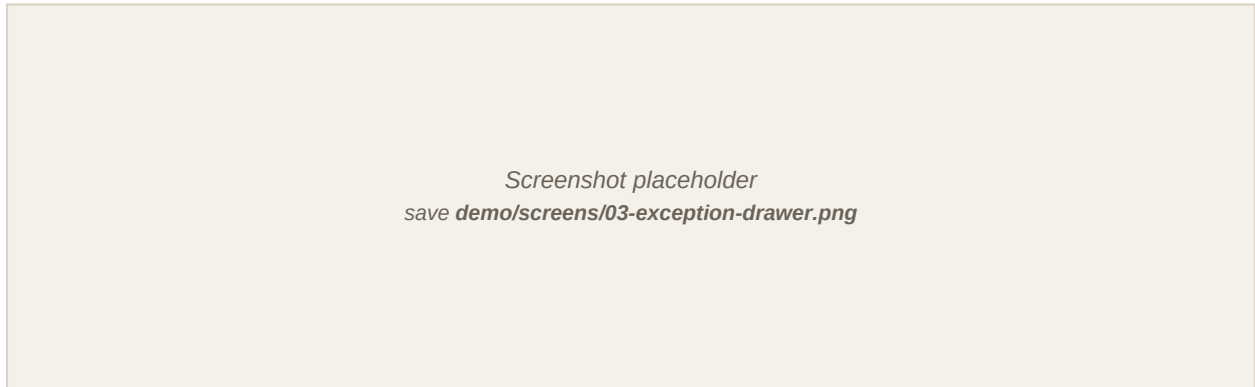
The agent ingests invoices, runs a deterministic three-way match (PO ↔ goods receipt ↔ invoice), and auto-clears the clean ones. The queue holds the exceptions — price variances, missing POs, duplicates, over-limit and tax-jurisdiction cases — each with a flag reason, amount at risk, confidence, and the agent that raised it.

Screenshot placeholder
save `demo/screens/02-exception-queue.png`

What the AP Matching agent couldn't safely auto-approve — the only invoices a human needs to touch.

2.3 Reviewing an exception

Opening an exception shows the PO/GR/invoice side-by-side with the mismatch highlighted, the line items, a confidence ring, and the contract/policy it's grounded in. The agent gives a recommendation (Accept in one click, or Override). **Explain with AI** drafts a controls-analyst narrative and a suggested disposition; the model badge shows which model produced it (local Ollama or hosted), and only the discrepancy — never the vendor master — is sent to it.



The detail drawer: 3-way match evidence, the agent's recommendation, and an AI-drafted explanation.

2.4 Cash Application (AR)

The Cash Application agent reconciles deposits against open AR invoices. High-confidence matches auto-apply; the rest — partial matches, unidentified ACH — come here for a human, pre-filled with the agent's proposed invoice set and confidence.

Screenshot placeholder
save **demo/screens/04-cash-application.png**

Incoming deposits matched to open invoices — exact, remittance-hint, then fuzzy.

2.5 Collections

The Collections agent scores each overdue account by days-past-due and balance, prioritizes them, and drafts the outreach message — but a person sends anything customer-facing. The risk distribution shows the book at a glance.

Screenshot placeholder
save **demo/screens/06-collections.png**

Risk-ranked overdue accounts with agent-drafted outreach (a human sends).

2.6 Anomalies & Leakage

The Anomaly agent watches the ledger and surfaces severity-ranked signals with the dollars at risk, plus a leakage-recovered trend — the ROI story for the program.

Screenshot placeholder
save **demo/screens/07-anomalies.png**

Where the money comes back: duplicate clusters, validation abuse, charge-capture faults, off-contract spend.

2.7 Agents console — autonomy

Each agent shows its pipeline (intake → match → decide → act → escalate), recent decisions with reasons and sources, and an editable confidence threshold. Agents start in **suggest-only** mode and are dialed toward auto-action; a global kill switch forces everything back to suggest-only in an incident. Changing autonomy requires the Controller role (segregation of duties).

Screenshot placeholder
save `demo/screens/08-agents.png`

Every agent in one place; promote each from suggest-only to auto as it earns trust.

2.8 Analytics

Touchless rate, cost per invoice, close-cycle time, DSO, and human-override rate over time — alongside a live panel that reflects this session's real agent activity. Promote an agent to auto and you watch the touchless rate climb and the open queue fall.

Screenshot placeholder
save `demo/screens/09-analytics.png`

Historical trends plus a live-this-session block computed from the audit trail.

2.9 Agent & Audit Trail

Every auto-action, human resolution, threshold change, and AI explanation — with the model that produced it — is appended here, never edited. Confidence thresholds are co-owned with the Controller. When an auditor asks 'why was this paid,' the answer is one click with the evidence attached.

Screenshot placeholder
save **demo/screens/10-audit.png**

The SOX story: an immutable, queryable log of every action and who took it.

2.10 Importing your data (CSV / ERP)

Finance OS reads real data, not just the bundled sample. Download a CSV template from the Exception Queue or Cash Application screen, fill in your invoices, POs, receipts, open invoices, or deposits, and import — the agents re-run on your numbers in seconds (duplicate invoices are auto-detected by vendor and amount). The same connector interface that reads a CSV today plugs into NetSuite, SAP, or QuickBooks tomorrow; bank feeds and OCR intake sit behind the same seam, and nothing is hard-wired to a vendor.

Appendix A · Technical architecture

Four layers, front to back, with a shared schema and an event/audit log cross-cutting all of them.

apps/web	React 19 + Vite + Zustand — the seven-screen UI (Espresso theme), typed against the shared schema
services/api	FastAPI BFF — read models + command endpoints, auth/RBAC, audit write-through, CSV ingestion
services/agents	Agent layer — deterministic engines (3-way match, cash match, risk scoring) + LLM reasoning, under confidence-threshold gates
packages/shared	Domain schema (types) shared by web and API
packages/connectors	Abstract ERP / bank / ingestion contracts + adapters (mock, file/CSV); vendor SDKs slot in here
persistence	stdlib sqlite3 — exceptions, deposits, collections, append-only audit, agent config (Postgres-ready behind repo.py)

The agent pipeline

Intake → Enrich & Match → Decide → (Auto-act | Escalate) → Audit. Clean cases are decided by rules; ambiguous ones get an LLM-drafted rationale. Confidence is compared to the agent's threshold: above it (and only when not in suggest-only mode) the agent auto-acts via a connector and logs; below it the item escalates to the analyst, pre-filled with the recommendation.

Security & control

- **RBAC** — Analyst (least-privilege) and Controller; tuning autonomy and resolving items $\geq$ \$20k require Controller (segregation of duties).
- **Immutable audit** — append-only; covers agent and human actions, including which model produced each AI explanation.
- **Secrets** via environment / secret manager, never in the repo; parameterized SQL; CORS allow-list; global kill switch.
- **Data minimization** — the LLM sees only the discrepancy; it can run fully local (Ollama / vLLM) so data never leaves your infrastructure.

Appendix B · What's built so far

Delivered (Phases 0–4 of the design spec, plus LLM reasoning and ingestion):

- **Foundation** — monorepo, shared schema, FastAPI, connector contracts + mock seam; all seven screens wired to the API.
- **AP exceptions** — deterministic tolerance-aware 3-way match engine; AP Matching agent (suggest-only); persistence + immutable audit.
- **AR cash application** — deposit→invoice match engine (exact / remittance / batch / fuzzy); apply flow.
- **Collections, Anomalies, Analytics** — risk-scored outreach drafts; severity-ranked leakage detection; analytics with a live-session block from the audit trail.
- **Autonomy** — promote an agent suggest-only → auto and it auto-acts on threshold-clearing items, moving the touchless rate.
- **Hardening** — Auth + RBAC with segregation of duties; security review (SECURITY.md).
- **Data in** — File/CSV ERP adapter + AP/AR/Collections CSV upload with templates and auto duplicate detection.
- **LLM-assisted reasoning** — swappable provider (local Ollama / hosted / offline narrator), data-minimized, audit-logged with the model used.
- **Quality** — 16 unit tests across the match engines; clean web build.

Roadmap (deferred — needs real credentials / infra)

- Real vendor-SDK ERP / bank / OCR adapters (the swap seam is a single file).
- Real identity provider + JWT, TLS / rate limiting, Postgres, async intake for volume, and deployment.

Finance OS / APAR — V2 research build. Generated from the live product.